Manual de Desarrollo Full Stack con Django y ReactJS

Este manual está diseñado para desarrolladores que deseen construir una API escalable con Django como backend y React.js para el frontend. Cada sección se ha elaborado para ofrecer un desarrollo progresivo del proyecto.

Versión: 1.0
Fecha: 30 de octubre de 2024
Autor: Javier Velazquez Villelas

Versión: 1.0.3

Contenido

Extensiones para Visual Studio Code (Sugerencia) 5

Creando el proyecto de Django para el backend 5

- Crear el proyecto de Django dentro del folder del bakend (cd backend) 5
- Crear el entorno virtual 5
- Activar el entorno virtual 5
- Instalando dJango 5
- Instalando Django Rest Framework 5
- Instalando drf-yasg 5
- Agregar DRF (Django Rest Framework) y drf_yasg al archivo de seetings 6
- Configurando la documentación de la API 6
- Creando superuser para el panel de Django 6
- Corriendo el servidor 6

Modificando el modelo del usuario 7

- Eliminar la base de datos (el archivo llamado db.sqlite3) 7
- Crear la aplicación de users 7
- Sustituir el Username por el Email en el login 8
- Mostrar aplicación en el área del Admin 8

Creando el CRUD de usuarios 9

Override de la creación del usuario 10

Override de la actualización del usuario 10

Endpoint para obtener los datos personales 11

Creando el endpoint de login con JWT en Django 12

Creando el proyecto de React 13

Instalando un UI React para la aplicación del frontend 13

Configurando SASS 14

Instalando React Router v6 15

Layout para las pantallas 16

Creando sistema de rutas con React Router Dom v6 19

Página de Login 21

Estado y validación del formulario de login 23

Realizando login y configurando CORS 24

Componente Toast para los errores 26

Versión: 1.0.3

Contexto del usuario y custom hook 28
Registro de usuarios 33
Backend 33
Frontend 35
Fixes 39
Error al ingresar con las credenciales de Django en Swagger 39
Referencias 40
Glosario 40

Tabla de Versiones

Versión	Fecha	Comentarios	Pagina
1.0.0	30-Oct-2024	Extensiones sugeridas para VSC, Conf. DRF y Doc. Gestión de usuarios, Endpoint personalizados y autenticación con JWT	5
1.0.1	06-Nov-2024	Creando el proyecto de React	13
1.0.2	17-Nov-2024	Contexto del Usuario y custom hook	28
1.0.3	26-Nov-2024	Registro de usuarios	33
1.0.3	26-Nov-2024	Fixes	39

Extensiones para Visual Studio Code (Sugerencia)

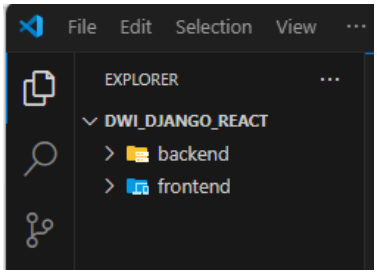
- *ES7 React/Redux/GraphQL/React-Native snippets*
- *Material Icon Theme*
- *Prettier - Code formatter*
- *Python*

Instalacion de Node *(En caso de no tenerlo instalado)*

Instalacion de Python *(En caso de no tenerlo instalado)*

Creando el proyecto de Django para el backend

Para crear este nuevo proyecto como se viene haciendo hasta ahora, se crea un folder principal y dentro se crean los directorios del frontend y backend. De la siguiente manera.



Crear el proyecto de Django dentro del folder del bakend (cd backend)

django-admin startproject restaurante . (Considerar el punto al final para no crear un subfolder de restaurante)

Crear el entorno virtual

python -m venv venv

Activar el entorno virtual

./venv/Scripts/activate

Instalando dJango

pip install django

python.exe -m pip install --upgrade pip

Instalando Django Rest Framework

https://www.django-rest-framework.org/#installation

pip install djangorestframework

Instalando drf-yasg

pip install drf-yasg

Agregar DRF (Django Rest Framework) y drf_yasg al archivo de seetings

```
# Application definition
BASE_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

THIRD_APPS = [
    'rest_framework',
    'drf_yasg',
]

OWN_APP = [

]

INSTALLED_APPS = BASE_APPS + THIRD_APPS + OWN_APP
```

Configurando la documentación de la API

Modificar el archivo urls del proyecto, como lo muestra la documentación oficial en el siguiente link.

https://drf-yasg.readthedocs.io/en/stable/readme.html#installation

Creando superuser para el panel de Django

python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser

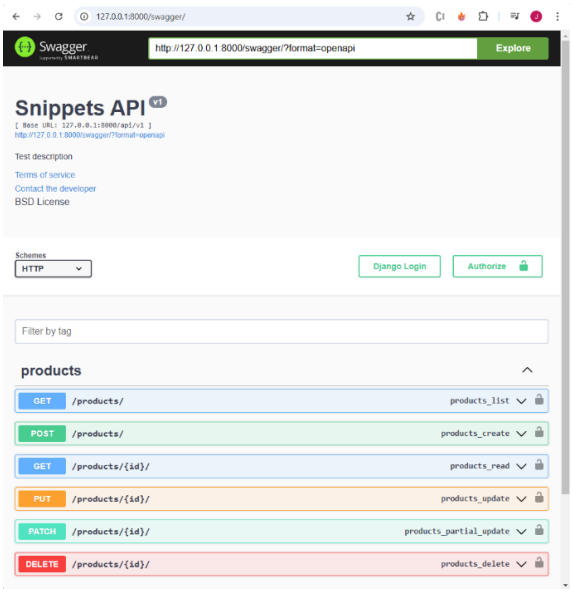
Corriendo el servidor

python manage.py runserver

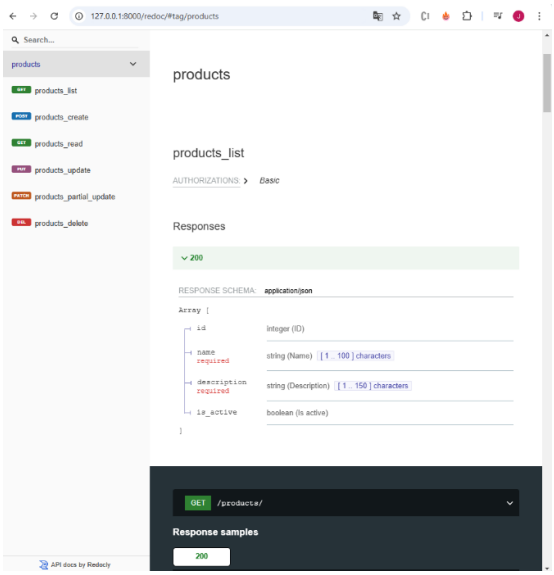
*NOTA: drf-yasg es una biblioteca para **Django Rest Framework** que facilita la generación de documentación de APIs en formato **OpenAPI** y la visualización interactiva de esta documentación a través de interfaces como **Swagger UI** o **ReDoc**. Es especialmente útil para proyectos que desean proporcionar documentación detallada y amigable para sus APIs sin escribirla manualmente, ya que genera gran parte del contenido automáticamente en función de los endpoints y serializadores definidos en la aplicación.*

Por el momento a ustedes no les mostrara aun nada porque no se ha creado aun una aplicación. Conforme agreguemos endpoint al proyecto, iran apareciendo.

http://127.0.0.1/swagger/?format=openapi



http://127.0.0.1/redoc



Modificando el modelo del usuario

Eliminar la base de datos (el archivo llamado db.sqlite3)

Crear la aplicación de users

Detener el servidor

python manage.py startapp users

Agregar la siguiente línea al final del archivo de setting.py

```
# USERS
AUTH_USER_MODEL = 'users.User'
```

Agregar aplicación de users en el archivo de settings.py

```
OWN_APPS = [
    'users'
]
```

Modificar el archivo de models.py en la aplicación de uses

```
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    pass
```

Versión: 1.0.3

Crear migraciones y el superusuario

python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser

Sustituir el Username por el Email en el login

modificar el archivo de models.py

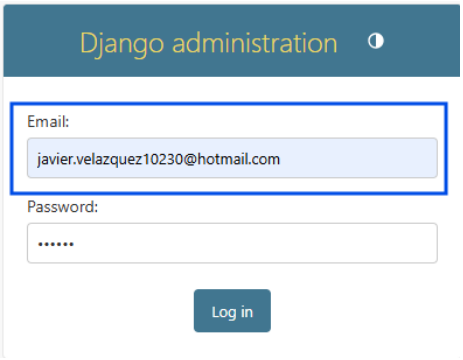
```
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
     email = models.EmailField(unique=True)

     USERNAME_FIELD = 'email'
     REQUIRED_FIELDS = []
```

python manage.py makemigrations
python manage.py migrate
python manage.py runserver

Ya se podrá utilizar el email para hacer el login



Mostrar aplicación en el área del Admin

Modificar el archivo de admin.py de la aplicación que se requiere mostrar en este caso de users

```
from users.models import User
# Register your models here.
admin.site.register(User)
```

Creando el CRUD de usuarios

Crear el archivo serializers.py en la aplicación de users

```
from rest_framework import serializers
from users.models import User
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'first_name', 'last_name', 'password', 'is_active', 'is_staff']
```

Modificar el archivo de views.py en la misma aplicación de users

```
from django.shortcuts import render
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAdminUser

from users.models import User
from users.serializers import UserSerializer

class UserApiViewSet(ModelViewSet):
    permission_classes = [IsAdminUser]
    serializer_class = UserSerializer
    queryset = User.objects.all()
```

Crear el archivo urls.py en la aplicación de users

```
from django.urls import path,include
from rest_framework import routers
from users import views

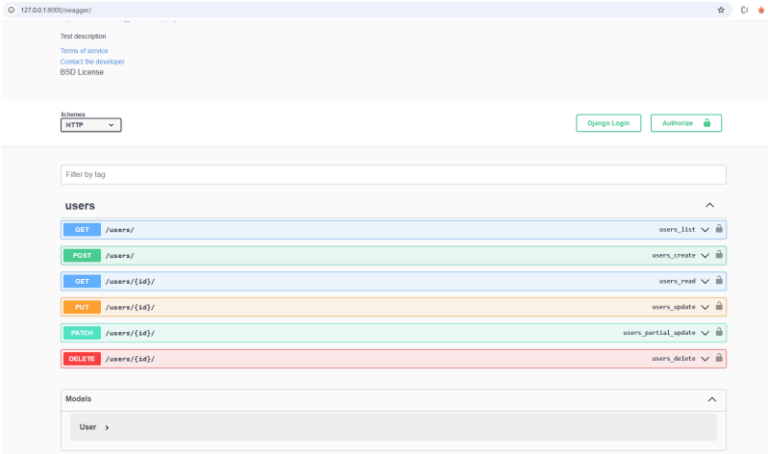
router = routers.DefaultRouter()
router.register(r'users', views.UserApiViewSet)

urlpatterns=[
    path('', include(router.urls))
]
```

Por último, modificar el archivo de urls.py del proyecto, solo agregar la línea del users

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/v1/', include('users.urls')),
    path('swagger<format>/', schema_view.without_ui(cache_timeout=0), name='schema-json'),
    path('swagger/', schema_view.with_ui('swagger', cache_timeout=0), name='schema-swagger-ui'),
    path('redoc/', schema_view.with_ui('redoc', cache_timeout=0), name='schema-redoc'),
]
```

Con esto ya tendríamos listo el CRUD de users en Django



Override de la creación del usuario

Modificar el archivo views.py de la aplicación de users

```
from django.shortcuts import render
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAdminUser
from django.contrib.auth.hashers import make_password

from users.models import User
from users.serializers import UserSerializer

class UserApiViewSet(ModelViewSet):
    permission_classes = [IsAdminUser]
    serializer_class = UserSerializer
    queryset = User.objects.all()

    def create(self, request, *args, **kwargs):
        request.data['password'] = make_password(request.data['password'])
        return super().create(request, *args, **kwargs)
```

Override de la actualización del usuario

Agregar la siguiente función al archivo de views.py de la aplicación de users

```
    def partial_update(self, request, *args, **kwargs):
        if "password" in request.data:
            request.data["password"] = make_password(request.data["password"])
        else:
            request.data["password"] = request.user.password
        return super().partial_update(request, *args, **kwargs)
```

Endpoint para obtener los datos personales

Modificar el archivo views.py de la aplicación de users

```python
from django.shortcuts import render
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAdminUser, IsAuthenticated
from rest_framework.views import APIView
from rest_framework.response import Response
from django.contrib.auth.hashers import make_password

from users.models import User
from users.serializers import UserSerializer

class UserApiViewSet(ModelViewSet):
    permission_classes = [IsAdminUser]
    serializer_class = UserSerializer
    queryset = User.objects.all()

    def create(self, request, *args, **kwargs):
        request.data['password'] = make_password(request.data['password'])
        return super().create(request, *args, **kwargs)

    def partial_update(self, request, *args, **kwargs):
        if "password" in request.data:
            request.data["password"] = make_password(request.data["password"])
        else:
            request.data["password"] = request.user.password
        return super().partial_update(request, *args, **kwargs)

class UserView(APIView):
    permission_classes = [IsAuthenticated]

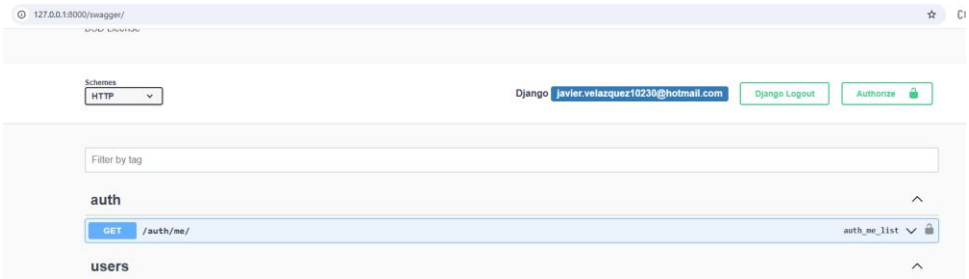
    def get(self, request):
        serializer = UserSerializer(request.user)
        return Response(serializer.data)
```

Modificar el archivo urls.py de la aplicación de users

```python
from django.urls import path,include
from rest_framework import routers
from users import views

router = routers.DefaultRouter()
router.register(r'users', views.UserApiViewSet)

urlpatterns=[
    path('', include(router.urls)),
    path('auth/me/', views.UserView.as_view())
]
```



Creando el endpoint de login con JWT en Django

Instalar Django Rest Framework SimpleJWT

https://django-rest-framework-simplejwt.readthedocs.io/en/latest/
pip install djangorestframework-simplejwt

Agregar lo siguiente en el archivo de settings.py del proyecto

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}
```

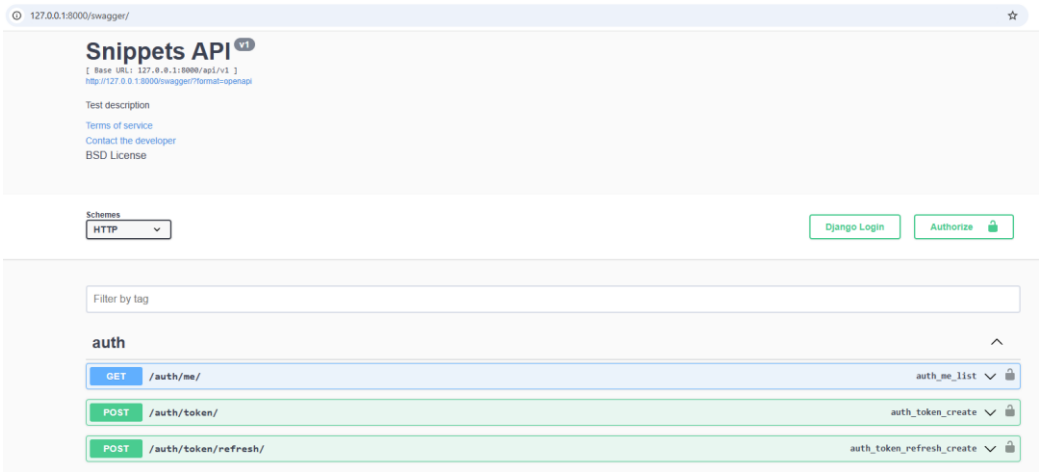
Modificar el archivo urls.py de la aplicación de users

```
from django.urls import path,include
from rest_framework import routers
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

from users import views

router = routers.DefaultRouter()
router.register(r'users', views.UserApiViewSet)

urlpatterns=[
    path('', include(router.urls)),
    path('auth/me/', views.UserView.as_view()),
    path('auth/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),  # Aqui pueden utilizar el
nombre de auth/token/  o  auth/login/
    path('auth/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```



Creando el proyecto de React

Para que esto funcione correctamente se tendrá que ejecutar la línea fuera del folder del backend

npm create vite@latest frontend
Seleccionar React y después Javascript
cd frontend
npm install
npm run dev

Instalando un UI React para la aplicación del frontend

Nota: Las bibliotecas de componentes de React UI son colecciones de elementos de interfaz de usuario listos para usar o precodificados que ayudan a acelerar el desarrollo de software.

https://prismic.io/blog/react-component-libraries
https://react.semantic-ui.com/modules/modal/

npm install semantic-ui-react semantic-ui-css

Eliminar los archivos App.css e index.css, emilinar los logos de react.svg y vite.svg

Modificar el archivo main.jsx (revisar la documentación de semantic-ou-react)

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

import "semantic-ui-css/semantic.min.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

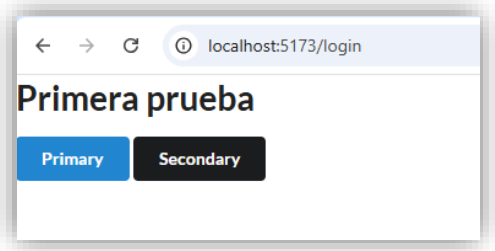
Modificar el archivo App.jsx

```jsx
import { Button } from "semantic-ui-react";

function App() {
  return (
    <>
      <h1>Primera prueba</h1>
      <div>
        <Button primary>Primary</Button>
        <Button secondary>Secondary</Button>
      </div>
    </>
  );
}

export default App;
```

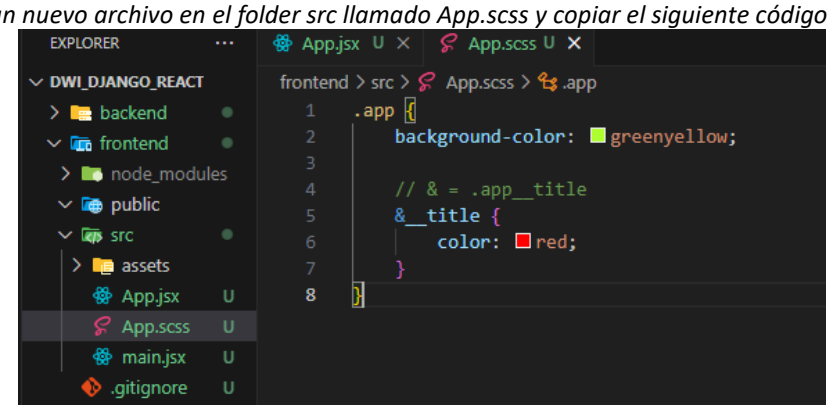
Aún falta darle más formato, pero tendríamos que tener algo ya similar a estos botones



Configurando SASS

npm add sass

Crear un nuevo archivo en el folder src llamado App.scss y copiar el siguiente código



Modificar el archivo App.jsx (Se importa el archivo scss y se agrega las className)

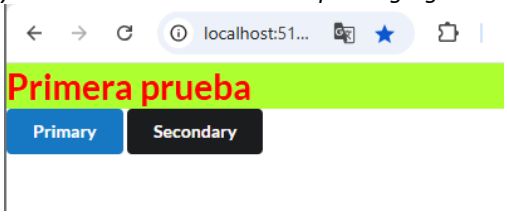
```jsx
import { Button } from "semantic-ui-react";
import "./App.scss";

function App() {
  return (
    <>
      <div className="app">
        <h1 className="app__title">Primera prueba</h1>
      </div>

      <div>
        <Button primary>Primary</Button>
        <Button secondary>Secondary</Button>
      </div>
    </>
  );
}

export default App;
```

Con eso ya tendríamos el archivo scss para agregar las clases customizadas.



Instalando React Router v6

npm install react-router-dom

Crear un nuevo folder en src llamado routes

Crear los siguientes archivos dentro del folder de routes y copiar el código en el archivo correspondiente.

index.jsx

```
export * from './Navigation'
```

Navigation.jsx

```
import React from "react";
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";

export function Navigation() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Navigate to="" />} />
      </Routes>
    </BrowserRouter>
  );
}
```

routes.admin.jsx

```
const routesAdmin = []

export default routesAdmin
```

routes.client.jsx

```
const routesClient = []

export default routesClient
```

routes.common.jsx

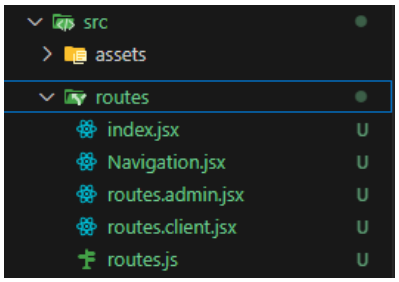
```
const routesCommon = []

export default routesCommon
```

routes.js

Nota: se pudiera usar solo un archivo de routes.js para administrar las rutas si el proyecto es pequeño, de lo contrario recomiendo hacer divisiones por segmentos, así se podrá mantener más ordenado el código y más fácil su mantenimiento.

Así se debería de ver el directorio de routes *(Falto el archivo de routes.common.jsx en l a imagen)*



Layout para las pantallas

Se va a crear un sistema de layouts para que la parte del administrador se vea de una manera y la parte del cliente se vea de otra manera.

Crear un nuevo folder en src llamado layouts que contengan los siguientes archivos.

AdminLayout.jsx

```jsx
export function AdminLayout({ children }) {
  return (
    <div>
      <p>AdminLayout</p>
      {children}
    </div>
  );
}
```

CommonLayout.jsx

```jsx
export function CommonLayout({ children }) {
  return (
    <div>
      <p>CommonLayout</p>

      {children}
    </div>
  );
}
```

ClientLayout.jsx

```jsx
export function ClientLayout({ children }) {
  return (
    <div>
      <p>ClientLayout</p>

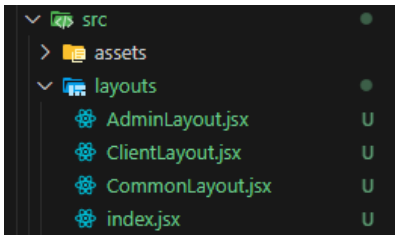
      {children}
    </div>
  );
}
```

Index.jsx

```jsx
export * from "./ClientLayout";
export * from "./AdminLayout";
export * from "./CommonLayout";
```

Nota: en cada folder creamos un archivo index.jsx que exporta todos los archivos de ese folder, esto ayudara al momento de importar muchos componentes que están en un mismo folder. Pero mas adelante lo estaremos viendo con ejemplos más claros.

Así se debería de ver el directorio de layouts



Crear un nuevo folder en src llamado pages, dentro de pages agregar tres nuevos folders llamados admin, client y common

Dentro de cada folder crear los siguientes archivos.

pages -> admin

index.jsx

```
export * from "./Dashboard";
```

Dashboard.jsx

```
import React from "react";

export function Dashboard() {
  return (
    <div>
      <h1>Dashboard Page</h1>
    </div>
  );
}
```

pages -> client

Home.jsx

```
import React from "react";

export function Home() {
  return (
    <div>
      <h1>Home Page</h1>
    </div>
  );
}
```

index.jsx

```
export * from "./Home";
```

pages -> common

index.jsx

```
export * from "./Login";
export * from "./Register";
export * from "./Error404";
```

Error404.jsx

```
import React from "react";

export default function Error404() {
  return (
    <div>
      <h1>Error 404 Page</h1>
    </div>
  );
}
```

Login.jsx

```
import React from "react";

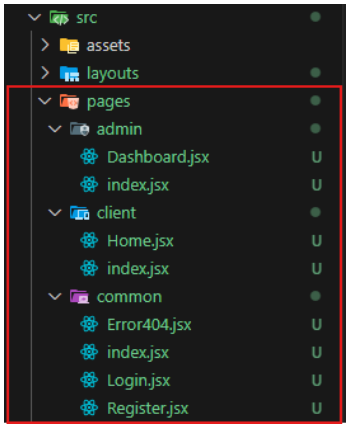
export function Login() {
  return (
    <div>
      <h1>Login Page</h1>
    </div>
  );
}
```

Register.jsx

```jsx
import React from "react";

export function Register() {
  return (
    <div>
      <h1>Register Page</h1>
    </div>
  );
```

Así se debería de ver el directorio de pages



Creando sistema de rutas con React Router Dom v6

Modificar el archivo routes.admin.jsx

```jsx
import { AdminLayout } from "../layouts";
import { Dashboard } from "../pages/admin";

const routesAdmin = [
  {
    path: "/dashboard",
    layout: AdminLayout,
    component: Dashboard,
  },
];

export default routesAdmin;
```

Modificar el archive routes.client.jsx

```jsx
import { ClientLayout } from "../layouts";
import { Home } from "../pages/client";

const routesClient = [
  {
    path: "/",
    layout: ClientLayout,
    component: Home,
  },
  {
    path: "/home",
    layout: ClientLayout,
    component: Home,
  },
];

export default routesClient;
```

Modificar el archivo routes.common.jsx

```jsx
import { CommonLayout } from "../layouts";
import { Login, Register, Error404 } from "../pages/common";

const routesCommon = [
  {
    path: "/login",
    layout: CommonLayout,
    component: Login,
  },
  {
    path: "/register",
    layout: CommonLayout,
    component: Register,
  },
  {
    path: "*",
    layout: CommonLayout,
    component: Error404,
  },
];

export default routesCommon;
```

Modificar el archivo routes.js

```jsx
import routesClient from "./routes.client";
import routesAdmin from "./routes.admin";
import routesCommon from "./routes.common";

export const routes = [
  ...routesAdmin,
  ...routesClient,
  ...routesCommon
]
```

Modificar el archivo Navigation.jsx

```jsx
import React from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { routes } from "./routes";

export function Navigation() {
  return (
    <BrowserRouter>
      <Routes>
        {routes.map((route, index) => {
          return (
            <Route
              key={index}
              path={route.path}
              element={
                <route.layout>
                  <route.component />
                </route.layout>
              }
            />
          );
        })}
      </Routes>
    </BrowserRouter>
  );
}
```

Modificar el archivo App.jsx

```jsx
import "./App.scss";
import { Navigation } from "./routes";

function App() {
  return (
    <>
      <div>
        <Navigation />
      </div>
    </>
  );
}

export default App;
```

Ya con esta configuración tenemos la división de rutas de React.
Dashboard se debería de encontrar en el AdminLayout, Login, Register y Error404 en CommonLayout y Home en ClientLayout. Como les comento, esto puede utilizar solo un archivo si el proyecto es pequeño.

Página de Login

Modificar el archivo AdminLayout.jsx

```jsx
import { Login } from "../pages/common";

export function AdminLayout({ children }) {
  const auth = null;

  if (!auth) return <Login />;

  return (
    <div>
      <p>AdminLayout</p>
      {children}
    </div>
  );
}
```

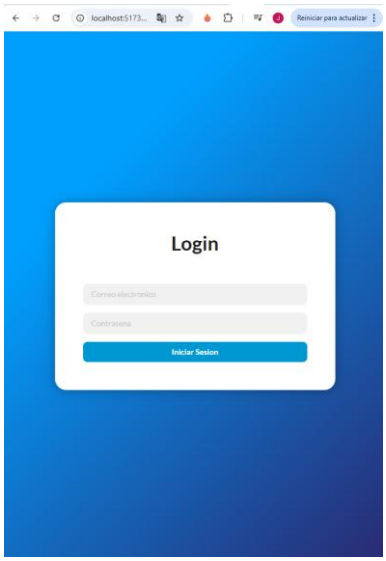
Modificar el archivo Login.jsx

```jsx
import React from "react";
import { Button, Form } from "semantic-ui-react";

export function Login() {
  return (
    <div className="login-admin">
      <div className="login-admin__content">
        <h1>Login</h1>

        <Form className="login-form-admin">
          <Form.Input name="email" placeholder="Correo electronico" />
          <Form.Input
            name="password"
            type="password"
            placeholder="Contrasena"
          />
          <Button type="submit" content="Iniciar Sesion" primary fluid />
        </Form>
      </div>
    </div>
  );
}
```

Nota: Para este proyecto estaremos agregando algo de código scss a los componentes para familiarizarnos con los conceptos, también vamos a sobre escribir las clases de los componentes ya existentes de Semantic UI.



Versión: 1.0.3

Modificar el archivo App.scss

```scss
/* LOGIN PAGEN START */
.login-admin {
    height: 100vh;
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: #2a2a72;
    background-image: linear-gradient(315deg, #2a2a72 0%, #009ffd 74%);

    &__content {
        background-color: #fff;
        padding: 50px 50px;
        border-radius: 20px;
        width: 500px;
        box-shadow: 0px 0px 20px 0px rgba(0, 0, 0, 0.3);
        -webkit-box-shadow: 0px 0px 20px 0px rgba(0, 0, 0, 0.3);
        -moz-box-shadow: 0px 0px 20px 0px rgba(0, 0, 0, 0.3);

        >h1 {
            font-size: 35px;
            text-align: center;
            margin-bottom: 50px;
        }
    }
}
/* LOGIN PAGE END*/

/* Override Semantic Components */
.ui.input {
    input {
        background-color: #f1f1f1 !important;
        border-color: #f1f1f1 !important;
        border-radius: 10px !important;

        &:focus {
            background-color: #fff !important;
            box-shadow: 0px 0px 10px 0px rgba(0, 0, 0, 0.2) !important;
            -webkit-box-shadow: 0px 0px 10px 0px rgba(0, 0, 0, 0.2) !important;
            -moz-box-shadow: 0px 0px 10px 0px rgba(0, 0, 0, 0.2) !important;

        }
    }
}

.ui.primary.button {
    background-color: #0098d3 !important;
    color: #fff;
    border-radius: 8px;
}

/* Override Semantic Components End*/
```

Estado y validación del formulario de login

Instalar los paquetes para la validación de formulario

npm install formik yup

Modificar el archivo Login.jsx

```jsx
import React from "react";
import { Button, Form } from "semantic-ui-react";
import { useFormik } from "formik";
import * as Yup from "yup";

export function Login() {
  const formik = useFormik({
    initialValues: initialValues(),
    validationSchema: validationSchema(),
    onSubmit: async (formValues) => {
      try {
        console.log("Valores enviados: ");
        console.log(formValues);
      } catch (error) {
        console.log(error.message);
      }
    },
  });

  return (
    <div className="login-admin">
      <div className="login-admin__content">
        <h1>Login</h1>

        <Form className="login-form-admin" onSubmit={formik.handleSubmit}>
          <Form.Input
            name="email"
            placeholder="Correo electrónico"
            value={formik.values.email}
            onChange={formik.handleChange}
            onBlur={formik.handleBlur}
            error={formik.errors.email}
          />
          <Form.Input
            name="password"
            type="password"
            placeholder="Contraseña"
            value={formik.values.password}
            onChange={formik.handleChange}
            error={formik.errors.password}
          />
          <Button type="submit" content="Iniciar Sesión" primary fluid />
        </Form>
      </div>
    </div>
  );
}

function initialValues() {
  return {
    email: "",
    password: "",
  };
}

function validationSchema() {
  return Yup.object({
    email: Yup.string()
      .email("Correo electrónico no válido")
      .required("El correo electrónico es obligatorio"),
    password: Yup.string().required("La contraseña es obligatoria"),
  });
}
```

Realizando login y configurando CORS

Crear un nuevo archivo (.env) dentro del folder de frontend y copiar las siguientes variables

```
VITE_BASE_API_URL=http://localhost:8000
VITE_LOCAL_STORAGE_ITEM_NAME=TOKEN
```

Crear un nuevo folder dentro de src llamado api (En este folder estará toda la lógica de comunicación entre el frontend y el backend)

Crear un nuevo archivo llamado user.js dentro del folder api y copiar lo siguiente

```
const API_URL = import.meta.env.VITE_BASE_API_URL

export async function loginApi (formData) {
    try {
        const url = `${API_URL}/api/v1/auth/token/`
        console.log('url', url)
        const params = {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(formData)
        }

        const response = await fetch(url, params)

        if (response.status !== 200) {
            throw new Error('Usuario o contraseña incorrecta')
        }

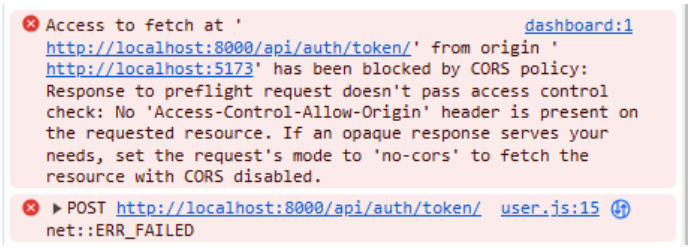
        const result = await response.json()
        return result
    } catch (error) {
        throw new Error('Error al iniciar sesión')
    }
}
```

Modificar el archivo Login.jsx (Identificar los cambios entre la versión anterior y esta, el archivo se adjunta incompleto)

```
import React from "react";
import { Button, Form } from "semantic-ui-react";
import { useFormik } from "formik";
import * as Yup from "yup";
import { loginApi } from "../../api/user";

export function Login() {
  const formik = useFormik({
    initialValues: initialValues(),
    validationSchema: validationSchema(),
    onSubmit: async (formValues) => {
      try {
        const response = await loginApi(formValues);
        const { access } = response;
        console.log(access);
      } catch (error) {
        console.log(error.message);
      }
    },
  });
```

Ya estamos haciendo la petición al servidor de Django, pero nos arroja un error de CORS.
Para solucionar esto, nos iremos a instalar un paquete a django y modificar el archivo de settings.py
https://pypi.org/project/django-cors-headers/



Abrir la terminal de comando en el directorio de backend

pip install django-cors-headers

Agregar el paquete 'corsheaders' en el archivo de settings.py

```
THIRD_APPS = [
    'rest_framework',
    'drf_yasg',
    'corsheaders',
]
```

Agregar el middleware como lo dice la documentación

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    "corsheaders.middleware.CorsMiddleware"
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Modificar la variable ALLOWED_HOSTS en settings.py

```
ALLOWED_HOSTS = ['127.0.0.1', "localhost"]
```

Por último, al final del archivo settings.py agregar las siguientes líneas para activar los CORS

```
CORS_ORIGIN_ALLOW_ALL = True
CORS_ALLOW_CREDENTIALS = True
```

Componente Toast para los errores

npm install react-toastify

Modificar el archivo main.jsx

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

import "semantic-ui-css/semantic.min.css";
import "react-toastify/ReactToastify.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Modificar el archivo App.jsx

```jsx
import "./App.scss";
import { ToastContainer } from "react-toastify";
import { Navigation } from "./routes";

function App() {
  return (
    <div>
      <Navigation />

      <ToastContainer
        position="bottom-center"
        autoClose={5000}
        hideProgressBar
        newestOnTop
        closeOnClick
        rtl={false}
        pauseOnFocusLoss
        draggable
        pauseOnHover={false}
      />
    </div>
  );
}

export default App;
```

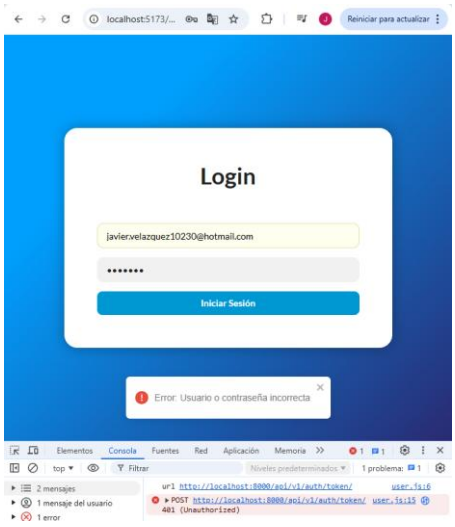
Modificar el archivo Login.jsx

```jsx
import React from "react";
import { Button, Form } from "semantic-ui-react";
import { useFormik } from "formik";
import * as Yup from "yup";
import { toast } from "react-toastify";
import { loginApi } from "../../api/user";

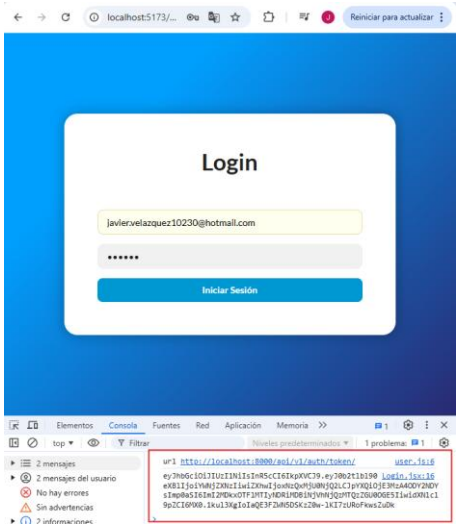
export function Login() {
  const formik = useFormik({
    initialValues: initialValues(),
    validationSchema: validationSchema(),
    onSubmit: async (formValues) => {
      try {
        const response = await loginApi(formValues);
        const { access } = response;
        console.log(access);
      } catch (error) {
        toast.error(error.message);
      }
    },
  });
```

Versión: 1.0.3

Ya se están mostrando los errores en el Login



En caso contrario, si se ingresan bien las credenciales retornara el Access token (Pueden revisar la información del token con esta url https://jwt.io/)



Versión: 1.0.3

Contexto del usuario y custom hook

Nota: En React, un contexto es una forma de compartir datos entre componentes, mientras que un hook es una función que permite "enganchar" el estado de React y el ciclo de vida
Contexto: Permite que un componente reciba información de un componente padre sin pasarla como props. Por ejemplo, un componente en el nivel más alto de una aplicación puede pasar el tema de interfaz de usuario a todos los componentes debajo.
Hook: Son funciones que permiten usar React sin clases. React proporciona algunos hooks incorporados, como useState, y también se pueden crear los propios.

Crear folder llamado context en src
Dentro de del folder context crear el archivo AuthContext.jsx

```jsx
import React, { useState, useEffect, createContext } from "react";

export const AuthContext = createContext({
  auth: undefined,
  login: () => null,
  logout: () => null,
});

export function AuthProvider({ children }) {
  const valueContext = {
    auth: null,
    login: () => console.log("Realizando login"),
    logout: () => console.log("Cerrando sesion"),
  };

  return (
    <AuthContext.Provider value={valueContext}>{children}</AuthContext.Provider>
  );
}
```

Modificar el archivo App.js

```jsx
import "./App.scss";
import { ToastContainer } from "react-toastify";
import { Navigation } from "./routes";
import { AuthProvider } from "./context/AuthContext";

function App() {
  return (
    <AuthProvider>
      <Navigation />

      <ToastContainer
        position="bottom-center"
        autoClose={5000}
        hideProgressBar
        newestOnTop
        closeOnClick
        rtl={false}
        pauseOnFocusLoss
        draggable
        pauseOnHover={false}
      />
    </AuthProvider>
  );
}

export default App;
```

Crear folder llamado hooks en src
Dentro del folder hooks crear el archivo useAuth.jsx

```jsx
import { useContext } from "react";
import { AuthContext } from "../context/AuthContext";

export const useAuth = () => useContext(AuthContext);
```

Crear un nuevo archivo llamada token.js en el folder de src/api

```
const TOKEN = import.meta.env.VITE_LOCAL_STORAGE_ITEM_NAME

export function setToken (token) {
  window.localStorage.setItem(TOKEN, token)
}

export function getToken () {
  return window.localStorage.getItem(TOKEN)
}

export function removeToken () {
  window.localStorage.removeItem(TOKEN)
}
```

Agregar la función de getMeApi en el archivo de user.js del folder api

```
const API_URL = import.meta.env.VITE_BASE_API_URL

export async function loginApi (formData) {
    try {
        const url = `${API_URL}/api/v1/auth/token/`
        console.log('url', url)
        const params = {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(formData)
        }

        const response = await fetch(url, params)

        if (response.status !== 200) {
            throw new Error('Usuario o contraseña incorrecta')
        }

        const result = await response.json()
        return result
    } catch (error) {
        throw new Error(error)
    }
}

export async function getMeApi(token) {
    try {
        const url = `${API_URL}/api/v1/auth/me/`

        const params = {
            headers: {
                Authorization: `Bearer ${token}`
            }
        }

        const response = await fetch(url, params)
        const result = await response.json()

        return result
    }catch(error) {
        throw error
    }
}
```

Dentro del folder hooks crear el archivo useUser.jsx

```jsx
import { getMeApi } from "../api/user";

export function useUser() {
  const getMe = async (token) => {
    try {
      const result = await getMeApi(token);
      return result;
    } catch (error) {
      throw new Error("Error al obtener el usuario");
    }
  };

  return { getMe };
}
```

Modificar el archivo AuthContext.jsx

```jsx
import React, { useState, useEffect, createContext } from "react";
import { setToken, getToken, removeToken } from "../api/token";
import { useUser } from "../hooks/useUser";

// 1. Crear el contexto
export const AuthContext = createContext({
  auth: undefined,
  login: () => null,
  logout: () => null,
});

// 2. Crear el provider
export function AuthProvider({ children }) {
  const [auth, setAuth] = useState(undefined);
  const { getMe } = useUser();

  useEffect(() => {
    const token = getToken();
    if (!token) {
      setAuth(null);
    } else {
      login(token);
    }
  }, []);

  const login = async (token) => {
    setToken(token);
    const me = await getMe(token);
    setAuth({ token, me });
  };

  const logout = () => {
    if (auth) {
      setAuth(null);
      setToken(null);
      removeToken();
    }
  };

  const valueContext = {
    auth,
    login,
    logout,
  };

  // Descomentar esta linea para que deje de verse el flash del login
  //if (auth === undefined) return null;

  return (
    <AuthContext.Provider value={valueContext}>{children}</AuthContext.Provider>
  );
}
```

Modificar el archivo Login.jsx

```jsx
import React from "react";
import { Button, Form, FormField } from "semantic-ui-react";
import { useFormik } from "formik";
import * as Yup from "yup";
import { toast } from "react-toastify";
import { loginApi } from "../../api/user";
import { useAuth } from "../../hooks/useAuth";
import { useNavigate } from "react-router-dom"; // Importa useNavigate

export function Login() {
  const { login } = useAuth();
  const navigate = useNavigate(); // Hook para redirección

  const formik = useFormik({
    initialValues: initialValues(),
    validationSchema: validationSchema(),
    onSubmit: async (formValues) => {
      try {
        const response = await loginApi(formValues);
        const { access } = response;
        login(access); // El access es el token que devuelve Django
        navigate("/dashboard"); // Redirige al Dashboard
      } catch (error) {
        toast.error(error.message);
      }
    },
  });

  return (
    <div className="login-admin">
      <div className="login-admin__content">
        <h1>Login</h1>

        <Form className="login-form-admin" onSubmit={formik.handleSubmit}>
          <label>Correo electrónico</label>
          <Form.Input
            name="email"
            placeholder="Correo electrónico"
            value={formik.values.email}
            onChange={formik.handleChange}
            onBlur={formik.handleBlur}
            error={formik.errors.email}
          />

          <label>Contraseña</label>
          <Form.Input
            name="password"
            type="password"
            placeholder="Contraseña"
            value={formik.values.password}
            onChange={formik.handleChange}
            error={formik.errors.password}
            autoComplete="false"
          />
          <Button type="submit" content="Iniciar Sesión" primary fluid />
        </Form>
      </div>
    </div>
  );
}

function initialValues() {
  return {
    email: "",
    password: "",
  };
}

function validationSchema() {
  return Yup.object({
    email: Yup.string()
      .email("Correo electrónico no válido")
      .required("El correo electrónico es obligatorio"),
    password: Yup.string().required("La contraseña es obligatoria"),
  });
}
```

Modificar el archivo AdminLayout.jsx

```jsx
import { Login } from "../pages/common";
import { useAuth } from "../hooks/useAuth";

export function AdminLayout({ children }) {
  const { auth } = useAuth();

  if (!auth) return <Login />;

  return <div>{children}</div>;
}
```

Modificar el archivo Dashboard.jsx

```jsx
import React from "react";
import { Button } from "semantic-ui-react";
import { useAuth } from "../../hooks/useAuth";

export function Dashboard() {
  const { logout, auth } = useAuth();
  const user = auth.me;

  return (
    <div>
      <h1>Dashboard Page</h1>
      <h3>Bienvenido {user.email}</h3>
      <Button type="submit" content="Cerrar Sesión" onClick={logout} />
    </div>
  );
}
```

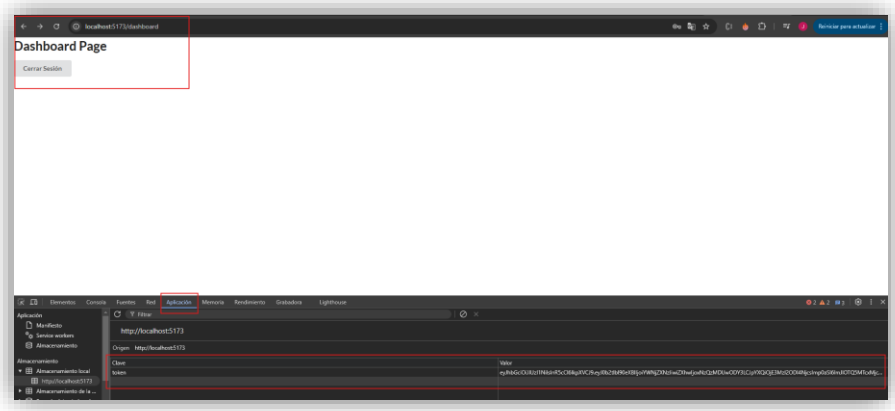
Modificar el archivo ClientLayout.jsx

```jsx
export function ClientLayout({ children }) {
  return <div>{children}</div>;
}
```

Modificar el archivo CommonLayout.jsx

```jsx
export function CommonLayout({ children }) {
  return <div>{children}</div>;
}
```

Con esto ya tenemos el Auth del usuario persistente.



Registro de usuarios

Backend

Modificar el archivo serializers.py en el folder backend/users

```python
from rest_framework import serializers
from users.models import User
from django.contrib.auth.hashers import make_password

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'first_name',
                  'last_name', 'password', 'is_active', 'is_staff']

class RegisterUserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        # Asegúrate de incluir estos campos
        fields = ['username', 'email', 'password']
        extra_kwargs = {
            # La contraseña no se devolverá en la respuesta
            'password': {'write_only': True},
        }

    def validate_password(self, value):
        if len(value) < 8:
            raise serializers.ValidationError(
                "La contraseña debe tener al menos 8 caracteres.")
        return value

    def create(self, validated_data):
        # Hashea la contraseña antes de guardar
        validated_data['password'] = make_password(validated_data['password'])
        return super().create(validated_data)
```

Modificar el archivo views.py en el folder de users

```python
from django.shortcuts import render
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAdminUser, IsAuthenticated
from rest_framework.views import APIView
from rest_framework.response import Response
from django.contrib.auth.hashers import make_password
from rest_framework import status
from rest_framework.permissions import AllowAny

from users.models import User
from users.serializers import UserSerializer, RegisterUserSerializer

class UserApiViewSet(ModelViewSet):
    permission_classes = [IsAdminUser]
    serializer_class = UserSerializer
    queryset = User.objects.all()

    def create(self, request, *args, **kwargs):
        request.data['password'] = make_password(request.data['password'])
        return super().create(request, *args, **kwargs)

    def partial_update(self, request, *args, **kwargs):
        if "password" in request.data:
            request.data["password"] = make_password(request.data["password"])
        else:
            request.data["password"] = request.user.password
        return super().partial_update(request, *args, **kwargs)

class UserView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        serializer = UserSerializer(request.user)
        return Response(serializer.data)

class RegisterUserView(ModelViewSet):
```

```
    permission_classes = [AllowAny]  # Permitir acceso sin autenticación
    serializer_class = RegisterUserSerializer
    queryset = User.objects.all()

    def post(self, request, *args, **kwargs):
        serializer = RegisterUserSerializer(data=request.user)
        if serializer.is_valid():
            user = serializer.save()
            return Response({
                'message': 'Usuario registrado exitosamente',
                'user': {
                    'username': user.username,
                    'email': user.email,
                }
            }, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Modificar el archivo urls.py de users

```
from django.urls import path, include
from rest_framework import routers
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

from users import views

router = routers.DefaultRouter()
router.register(r'users', views.UserApiViewSet)

urlpatterns = [
    path('', include(router.urls)),
    path('auth/me/', views.UserView.as_view()),
    path('auth/register/',
         views.RegisterUserView.as_view({'post': 'create'}), name='register'),
    # Aqui pueden utilizar el nombre de auth/token/  o  auth/login/
    path('auth/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('auth/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```

Con esto crearemos un nuevo endpoint de register que no ocupe estar autenticado el usuario.

Frontend

Modificar el archivo users.js en el folder frontend/src/api

```
const API_URL = import.meta.env.VITE_BASE_API_URL

export async function loginApi (formData) {
    try {
        const url = `${API_URL}/api/v1/auth/token/`
        console.log('url', url)
        const params = {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(formData)
        }

        const response = await fetch(url, params)

        if (response.status !== 200) {
            throw new Error('Usuario o contraseña incorrecta')
        }

        const result = await response.json()
        return result
    } catch (error) {
        throw new Error(error)
    }
}

export async function registerApi (formData) {
    try {
        const url = `${API_URL}/api/v1/auth/register/`
        console.log('url', url)
        const params = {
            method: 'POST',
            headers: {
                'accept': 'application/json',
                'Content-Type': 'application/json',
            },
            body: JSON.stringify(formData)
        }

        const response = await fetch(url, params)

        console.log(response)

        if (response.status !== 201) {
            throw new Error('No se puede crear el usuario')
        }

        const result = await response.json()
        return result
    } catch (error) {
        throw new Error(error)
    }
}

export async function getMeApi(token) {
    try {
        const url = `${API_URL}/api/v1/auth/me/`

        const params = {
            headers: {
                Authorization: `Bearer ${token}`
            }
        }

        const response = await fetch(url, params)
        const result = await response.json()

        return result
    }catch(error) {
        throw error
    }
}
```

Modificar el archivo de Register.jsx que se encuentra en el folder src/pages/common

```jsx
import React from "react";
import { Button, Form } from "semantic-ui-react";
import { useFormik } from "formik";
import * as Yup from "yup";
import { toast } from "react-toastify";
import { registerApi } from "../../api/user";
import { useNavigate } from "react-router-dom"; // Importa useNavigate

export function Register() {
  const navigate = useNavigate(); // Hook para redirección
  const formik = useFormik({
    initialValues: initialValues(),
    validationSchema: validationSchema(),
    onSubmit: async (formValues) => {
      try {
        const response = await registerApi(formValues);
        const { username } = response;
        navigate("/login"); // Redirige al Dashboard
      } catch (error) {
        toast.error(error.message);
      }
    },
  });
  return (
    <div className="register-admin">
      <div className="register-admin__content">
        <h1>Registro</h1>

        <Form className="register-form-admin" onSubmit={formik.handleSubmit}>
          <label>Username</label>
          <Form.Input
            name="username"
            placeholder="Username"
            value={formik.values.username}
            onChange={(e) => {
              const value = e.target.value.replace(/\s+/g, ""); // Elimina espacios en tiempo real
              formik.setFieldValue("username", value);
            }}
            onBlur={formik.handleBlur}
            error={formik.errors.username}
          />
          <label>Correo electrónico</label>
          <Form.Input
            name="email"
            placeholder="Correo electrónico"
            value={formik.values.email}
            onChange={formik.handleChange}
            onBlur={formik.handleBlur}
            error={formik.errors.email}
          />
          <label>Contraseña</label>
          <Form.Input
            name="password"
            type="password"
            placeholder="Contraseña"
            value={formik.values.password}
            onChange={formik.handleChange}
            error={formik.errors.password}
            autoComplete="false"
          />
          <label>Confirmar Contraseña</label>
          <Form.Input
            name="password_confirm"
            type="password"
            placeholder="Confirmar Contraseña"
            value={formik.values.password_confirm}
            onChange={formik.handleChange}
            error={formik.errors.password_confirm}
            autoComplete="false"
          />
          <Button type="submit" content="Iniciar Sesión" primary fluid />
        </Form>
      </div>
    </div>
  );
}
```

```
function initialValues() {
  return {
    username: "",
    email: "",
    password: "",
    password_confirm: "",
  };
}

function validationSchema() {
  return Yup.object({
    username: Yup.string()
      .required("El username es obligatorio")
      .matches(/^\S*$/, "El username no debe contener espacios"), // Prohíbe espacios,
    email: Yup.string()
      .email("Correo electrónico no válido")
      .required("El correo electrónico es obligatorio"),
    password: Yup.string()
      .required("La contraseña es obligatoria")
      .min(8, "La contraseña debe tener al menos 8 caracteres")
      .matches(
        /[A-Z]/,
        "La contraseña debe contener al menos una letra mayúscula"
      )
      .matches(/\d/, "La contraseña debe contener al menos un número")
      .matches(
        /[@$!%*?&#]/,
        "La contraseña debe contener al menos un carácter especial (@$!%*?&#)"
      ),
    password_confirm: Yup.string()
      .required("La confirmacion de contraseña es obligatoria")
      .oneOf([Yup.ref("password"), null], "Las contraseñas no coinciden"),
  });
}
```

Modificar el archivo App.scss

```
/* LOGIN & REGISTER PAGEN START */
.login-admin,
.register-admin {
    height: 100vh;
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: #2a2a72;
    background-image: linear-gradient(315deg, #2a2a72 0%, #009ffd 74%);

    &__content {
        background-color: #fff;
        padding: 50px 50px;
        border-radius: 20px;
        width: 500px;
        box-shadow: 0px 0px 20px 0px rgba(0, 0, 0, 0.3);
        -webkit-box-shadow: 0px 0px 20px 0px rgba(0, 0, 0, 0.3);
        -moz-box-shadow: 0px 0px 20px 0px rgba(0, 0, 0, 0.3);

        >h1 {
            font-size: 35px;
            text-align: center;
            margin-bottom: 50px;
        }
    }
}

.register-admin {
    color:blue;
}
/* LOGIN & REGISTER PAGE END*/

/* Override Semantic Components */
.ui.input {
    input {
        background-color: #f1f1f1 !important;
        border-color: #f1f1f1 !important;
```

```
        border-radius: 10px !important;

        &:focus {
            background-color: #fff !important;
            box-shadow: 0px 0px 10px 0px rgba(0, 0, 0, 0.2) !important;
            -webkit-box-shadow: 0px 0px 10px 0px rgba(0, 0, 0, 0.2) !important;
            -moz-box-shadow: 0px 0px 10px 0px rgba(0, 0, 0, 0.2) !important;

        }
    }
}

.ui.primary.button {
    background-color: #0098d3 !important;
    color: #fff;
    border-radius: 8px;
}

/* Override Semantic Components End*/
```

Fixes

Error al ingresar con las credenciales de Django en Swagger

Modificar el archivo de seetings.py en el backend

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
        'rest_framework.authentication.SessionAuthentication',
    )
}
```

Referencias

https://www.django-rest-framework.org/#installation

https://drf-yasg.readthedocs.io/en/stable/readme.html#installation

https://sqliteonline.com/

https://django-rest-framework-simplejwt.readthedocs.io/en/latest/

https://prismic.io/blog/react-component-libraries

https://pypi.org/project/django-cors-headers/

https://jwt.io/

Glosario

DWI (Desarrollo Web Integral)
VSC (Visual Studio Code)
DRF (Django Rest Framework)
JWT (JSON Web Token)
API (Application Programming Interface)
UI (user interface)